Revisão de Estrutura de Dados em C

Guia Prático de Ponteiros, Alocação de Memória e Listas Encadeadas

Ponteiros armazenam endereços de memória e permitem acesso direto às variáveis.

Um ponteiro é uma variável cujo valor é o endereço de memória de outra variável. Dominar ponteiros é essencial para manipulação eficiente de memória em C.

Operadores Fundamentais:

- **& (Operador de Endereço):** Retorna o endereço de memória exato onde uma variável está alocada.
- *** (Operador de Dereferenciação):** Acessa ou modifica o valor armazenado no endereço apontado pelo ponteiro.

> *Compreender essa mecânica de acesso indireto é o primeiro passo para construir estruturas de dados complexas, onde os elementos exigem referências explícitas de localização.*

Exemplo Prático em C:

```c
int numero = 42; // Declara um inteiro
int *ponteiro = &numero; // Armazena o endereço

// Modificando o valor original via ponteiro
*ponteiro = 99;
// Agora, o valor de 'numero' é 99
```

Visualização no Mapa de Memória:



Alocação Dinâmica com malloc e free

A alocação dinâmica reserva espaço na Heap que deve ser explicitamente liberado.

Pilha (Stack) vs. Heap

A **Stack** é estática, rápida e gerenciada automaticamente pelo compilador, mas possui tamanho fixo. A **Heap** é dinâmica, gerenciada pelo programador e ideal para estruturas de dados flexíveis que crescem durante a execução do programa.

Gerenciamento de Memória

`malloc()` aloca o tamanho especificado em bytes na Heap e retorna um ponteiro `void*`. É crucial verificar se o retorno é `NULL` (falha de alocação por falta de memória) antes do uso.

```c
// Alocação dinâmica de array com verificação de segurança
int *array = (int*) malloc(5 * sizeof(int));
if (array == NULL) { /* Tratamento de erro: falha */ return 1; }
// ... uso do array ...
free(array); // Liberação obrigatória ao final
```

⚠ **CUIDADO: Esquecer o `free()` gera Vazamento de Memória (Memory Leak), esgotando permanentemente os recursos do sistema.**

Listas Encadeadas Simples

Conexão estruturada de nós e alocação dinâmica na Heap

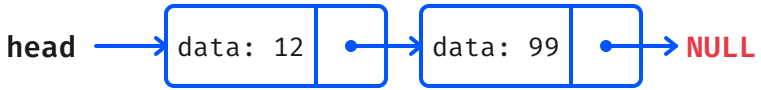
Diferente dos arrays, que exigem alocação contígua (blocos sequenciais de memória), as listas encadeadas distribuem seus elementos (nós) aleatoriamente pela **Heap**. Cada nó funciona como uma estrutura isolada conectada por ponteiros.

Elementos Críticos da Lista

- **Ponteiro head (Cabeça):** Um ponteiro fundamental que guarda o endereço do primeiro nó. Sem ele, toda a lista é perdida na memória.
- **Terminador NULL:** O ponteiro next do último nó deve obrigatoriamente apontar para NULL, servindo como condição de parada universal.

```c
struct Node {
    int data; // Armazena a informação
    struct Node* next; // Ponteiro auto-referenciado
};
```

Mapeamento Lógico na Memória



A manipulação de ponteiros na inserção e remoção exige a preservação do encadeamento.

Modificar uma lista encadeada requer cuidado cirúrgico com a ordem das operações de ponteiros para não quebrar a corrente.

Lógica de Inserção no Início (Push)

1. Alocar memória para o novo nó dinamicamente.
2. Inserir o dado no novo nó.
3. Apontar o `next` do novo nó para o atual `head`.
4. Atualizar o `head` para apontar para o novo nó.

Lógica de Remoção Básica

Para remover, deve-se localizar o nó anterior ao alvo, ajustar seu ponteiro `next` para pular o nó que será removido (apontando para `alvo->next`), e então usar `free()` no nó alvo para devolver a memória ao sistema.

```c
void inserirNoInicio(
   struct Node** head_ref,
   int novo_dado
) {
   struct Node* novo_no =
      (struct Node*) malloc(sizeof(struct Node));
   novo_no->data = novo_dado;
   // Conecta ao resto da lista
   novo_no->next = (*head_ref);
   // Atualiza a cabeça
   (*head_ref) = novo_no;
}
```

Listas Duplamente Encadeadas

Percurso bidirecional na estrutura ao custo de ponteiros extras

Uma evolução da lista simples é a lista duplamente encadeada, onde cada nó mantém conhecimento tanto do seu sucessor quanto do seu antecessor.

VANTAGENS

- Permite travessia nos dois sentidos (de frente para trás e de trás para frente).
- Remoção O(1) de nó específico conhecido: sem varreduras, pois o anterior está acessível via `prev`.

CUSTOS & RISCOS

- Maior consumo de memória: cada nó exige espaço adicional para armazenar o ponteiro `prev`.
- Complexidade algorítmica: inserção e remoção exigem múltiplos ponteiros atualizados.

node.h

```c
struct Node {
  int data;
  struct Node* next; // Aponta para o nó seguinte
  struct Node* prev; // Aponta para o nó anterior
};
```

EXERCÍCIO PRÁTICO 01

Busca e Contagem de Nós em Listas Simples

ENUNCIADO DO PROBLEMA

Escreva uma função em C que receba a cabeça (**head**) de uma lista encadeada simples e um valor inteiro.

A função deve percorrer a lista e retornar a quantidade de vezes que esse valor específico aparece nos nós.

ANÁLISE DE COMPLEXIDADE

O algoritmo possui complexidade de tempo O(n), pois no pior caso, precisamos visitar cada nó exatamente uma vez. A complexidade de espaço é O(1), pois usamos apenas variáveis auxiliares.

O(n)
Tempo (Linear)

O(1)
Espaço (Constante)

EXERCÍCIO PRÁTICO 01

Busca e Contagem de Nós em Listas Simples

ENUNCIADO DO PROBLEMA

Escreva uma função em C que receba a cabeça (**head**) de uma lista encadeada simples e um valor inteiro.

A função deve percorrer a lista e retornar a quantidade de vezes que esse valor específico aparece nos nós.

ANÁLISE DE COMPLEXIDADE

O algoritmo possui complexidade de tempo O(n), pois no pior caso, precisamos visitar cada nó exatamente uma vez. A complexidade de espaço é O(1), pois usamos apenas variáveis auxiliares.

O(n)
Tempo (Linear)

O(1)
Espaço (Constante)

resolucao.c

```c
int buscarEContar(struct Node* head, int valor_buscado) {
  int contador = 0;
  struct Node* atual = head; // Auxiliar
  // Percorre a lista até NULL
  while (atual ! = NULL) {
    if (atual→data == valor_buscado) {
      contador++; // Incrementa
    }
    atual = atual→next; // Avança
  }
  return contador;
}
```

Exercício 2: Inversão de lista duplamente encadeada

ajustando os ponteiros prev e next.

Enunciado

Escreva uma função que inverta completamente a ordem dos elementos de uma lista duplamente encadeada.

Lógica do Algoritmo

- Para inverter a lista, basta percorrer todos os nós.
- Para cada nó, trocar (swap) os ponteiros `next` e `prev`.
- O nó que era o último se tornará a nova cabeça da lista.

Exercício 2: Inversão de lista duplamente encadeada

ajustando os ponteiros prev e next.

Enunciado

Escreva uma função que inverta completamente a ordem dos elementos de uma lista duplamente encadeada.

Lógica do Algoritmo

- Para inverter a lista, basta percorrer todos os nós.
- Para cada nó, trocar (swap) os ponteiros **`next`** e **`prev`**.
- O nó que era o último se tornará a nova cabeça da lista.

C CODE

```c
void inverterLista(struct Node** head_ref) {
  struct Node* temp = NULL;
  struct Node* atual = *head_ref;
  // Troca next e prev para todos os nós da lista
  while (atual ! = NULL) {
    temp = atual→prev;
    atual→prev = atual→next;
    atual→next = temp;
    atual = atual→prev; // Avança para o próximo nó (que agora está em prev)
  }
  // Verifica se a lista estava vazia ou tinha apenas um nó
  if (temp ! = NULL) {
    *head_ref = temp→prev; // Atualiza a cabeça para o novo primeiro nó
  }
}
```